

YILDIZ TECHNICAL UNIVERSITY
APPLIED SQL (MTM4692)

Textile Warehouse Stock
Management System

Team No: 32

Adil Topçu 21052036

Mert Kılıç 21058031

Github Link: <https://github.com/mertkilicytd/textile-warehouse-stock-management-system>

Video Link:

https://drive.google.com/file/d/17K6Pj_9K0w2cLW7UqGSvNxvm7xl_Xy_S/view?usp=drive_link

1. INTRODUCTION

1.1 Problem Statement

In contemporary textile supply chains, the synchronization of incoming manufacturing goods and outgoing distribution is frequently decoupled by legacy manual logging. Legacy inventory methodologies rely on physical ledgers or fragmented flat spreadsheets, presenting catastrophic operational risks. These vulnerabilities include inconsistent stock level tallies, untracked modifications, latency in order replenishment pipelines, and data corruption stemming from multi-user race conditions.

The complexity scales non-linearly within the textile vertical. A single product line (e.g., a specific jacket design) expands into a multi-dimensional matrix of unique stock-keeping variations, spanning countless permutations of sizes (e.g., S, M, L, XL) and colors. Failing to isolate and trace transactions at this granular variation level results in asymmetric inventory situations, where high-demand variations suffer from invisible stockouts while stale variations accumulate holding costs unchecked.

1.2 The Proposed Solution

This project introduces a highly robust, secure, and transactional web-based Relational Database

Management System (RDBMS) architecture explicitly optimized for textile warehouse logisticians. By abstracting relational constraints into a lightweight Python/Flask presentation layer powered by an optimized SQLite transactional backend, the system eliminates human logging errors. Warehouse operatives interact with a sanitized, authenticated user interface that translates operational behaviors into atomic, schema-validated SQL transactions.

1.3 Project Objective and Justification

The core objective is to replace descriptive recording with automated, deterministic constraints that preserve historical records and real-time ledger states. The business logic prevents sub-zero stock counts, enforces cascading updates, map suppliers to granular item catalogs, and logs an indelible chronological trail of entries and exits linked to individual employees. This architectural approach improves operational efficiency, eliminates

human processing latency, and establishes a foundational data warehouse layer ready for predictive replenishment analytics.

2. RELATIONAL DATABASE DESIGN & ARCHITECTURE

The relational backend is constructed around six core entities designed to satisfy Third Normal Form (3NF)

requirements, eliminating transactional anomalies and redundant data fields.

2.1 Database Schema Reference & Entities

The operational layer utilizes the following specialized relations to segment administrative, logistical, and

product catalogs:

Table Name	Primary / Foreign Keys	Functional Responsibility & Role
------------	------------------------	----------------------------------

employees	employee_id (PK)	
-----------	------------------	--

Stores unique corporate identities, cryptographic credentials, and access tokens for system auditing.

categories	category_id (PK)	
------------	------------------	--

Enforces canonical categorization groupings (e.g., T-Shirts, Pants, Outerwear) to normalize product groupings.

suppliers	supplier_id (PK)	
-----------	------------------	--

Maintains critical logistical contacts, addresses, and telecom metadata for supply chain mapping.

products

product_id (PK)

category_id (FK)

supplier_id (FK)

The generalized product index that links distinct structural brands and descriptions to specific categories and B2B suppliers.

product_variants

variant_id (PK)

product_id (FK)

The multi-variant inventory table resolving granular color and size options, housing the exact real-time physical stock counts.

stock_movements

movement_id (PK)

variant_id (FK)

employee_id (FK)

The chronological immutable transaction ledger mapping every warehouse addition and subtraction back to an authoritative user.

2.2 Architectural Entity Relationship Definition

Data integrity is established through rigid referential constraints. The products table abstracts general

product data, eliminating the need to repeat text strings like brand name or descriptions across hundreds of unique

items. Instead, the product_variants table maintains a clean many-to-one mapping back to the parent item,

establishing a highly normalized structure. Physical movements do not modify variant numbers out of context;

instead, they generate a permanent record inside stock_movements, ensuring audit compliance and historical traceability.

3. TECHNICAL IMPLEMENTATION & SQL CONSTRAINTS

3.1 Data Definition Language (DDL) Schema Blueprints

The structural constraints, indexes, and mathematical checking limits are declared natively within the database

engine via the following SQL definitions:

```
PRAGMA foreign_keys = ON;
```

```
CREATE TABLE IF NOT EXISTS employees (  
  employee_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  full_name TEXT NOT NULL,  
  username TEXT NOT NULL UNIQUE,  
  password TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS categories (  
  category_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  category_name TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS suppliers (  
  supplier_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  supplier_name TEXT NOT NULL,  
  phone TEXT,
```

email TEXT,
address TEXT

);

CREATE TABLE IF NOT EXISTS products (
product_id INTEGER PRIMARY KEY AUTOINCREMENT,
product_name TEXT NOT NULL,
category_id INTEGER NOT NULL,
supplier_id INTEGER NOT NULL,
brand TEXT,
price REAL NOT NULL DEFAULT 0,
FOREIGN KEY (category_id) REFERENCES
categories(category_id),
FOREIGN KEY (supplier_id) REFERENCES
suppliers(supplier_id)

);

CREATE TABLE IF NOT EXISTS product_variants (
variant_id INTEGER PRIMARY KEY AUTOINCREMENT,
product_id INTEGER NOT NULL,
size TEXT NOT NULL,
color TEXT NOT NULL,
stock_quantity INTEGER NOT NULL DEFAULT 0
CHECK(stock_quantity >= 0),
FOREIGN KEY (product_id) REFERENCES
products(product_id),


```
UNIQUE(product_id, size, color)
);

CREATE TABLE IF NOT EXISTS stock_movements (
movement_id INTEGER PRIMARY KEY AUTOINCREMENT,
variant_id INTEGER NOT NULL,
employee_id INTEGER NOT NULL,
movement_type TEXT NOT NULL CHECK(movement_type IN
('ENTRY', 'EXIT')),
quantity INTEGER NOT NULL CHECK(quantity > 0),
movement_date TEXT NOT NULL DEFAULT
CURRENT_TIMESTAMP,
```

3.2 Advanced Mathematical Business Rules & Triggers

To enforce strict database level consistency, several data protection mechanisms are embedded directly into the relations:

Sub-Zero Invalidation: The variant matrix utilizes a mathematical rule $stock_quantity \geq 0$ inside the database. This rule blocks any attempts to remove stock that would result in an impossible negative quantity.

Granular Uniqueness Rule: The database enforces a `UNIQUE(product_id, size, color)`

constraint. This prevents conflicting duplicate variants from skewing stock counts and guarantees clean data.

Operational Signs Rule: The system enforces quantity > 0 on stock modifications, preventing zero or negative values from corrupting historical ledger data.

State Enumeration: The system enforces an explicit domain limit on transactions using a strict

CHECK(movement_type IN ('ENTRY', 'EXIT')) filter.

3.3 Advanced Logistical SQL Queries

Query 1: Dynamic Low Stock Detection with Multi-Table Joins

This query identifies critical depletion states where stock falls below a specific threshold ($Q_{\text{threshold}} <$

10), pulling descriptive details across tables to notify purchasing agents:

Query 2: Atomic Inventory Adjustments via Double Actions

When stock shifts, the system logs a permanent movement entry and updates the current stock level within a safe transaction block:

```
FOREIGN KEY (variant_id) REFERENCES  
product_variants(variant_id),
```

```
FOREIGN KEY (employee_id) REFERENCES  
employees(employee_id)
```

```
);
```

-

```
SELECT p.product_name, p.brand, c.category_name, pv.size,  
pv.color,  
pv.stock_quantity, s.supplier_name  
FROM product_variants pv  
JOIN products p ON pv.product_id = p.product_id  
JOIN categories c ON p.category_id = c.category_id  
JOIN suppliers s ON p.supplier_id = s.supplier_id  
WHERE pv.stock_quantity < 10  
ORDER BY pv.stock_quantity ASC;
```

4. APPLICATION ARCHITECTURE & INTEGRATION

4.1 Technology Stack Definition

The management suite is constructed using a robust technology stack designed to handle modern multi-user demands:

Database Engine: SQLite embedded RDBMS, configured with `PRAGMA foreign_keys = ON` to ensure

full database constraints during processing.

Application Server Framework: Flask (Python 3 base) handles web routing, parameter parsing, and maps

requests to safe parameters.

Cryptographic Security Layer: Secure credential management via `werkzeug.security` using adaptive

Argon2/PBKDF2 algorithms to protect user passwords.

4.2 Transaction Integrity and Resilience

The application code is structured to handle database errors gracefully. Rather than risking database

corruption or unexpected application crashes during invalid operations, the application uses explicit try-except

blocks to catch exceptions early. For example, if a user attempts to delete a supplier linked to active products, the code catches the error and provides clear feedback:

-- Step A: Write the permanent historical ledger record

```
INSERT INTO stock_movements (variant_id, employee_id,
movement_type, quantity,
movement_date)
```

```
VALUES (14, 2, 'ENTRY', 75, CURRENT_TIMESTAMP);
```

-- Step B: Update the current variation table value

```
UPDATE product_variants
```

```
SET stock_quantity = stock_quantity + 75
```

```
WHERE variant_id = 14;
```

-

```
@app.route("/suppliers/<int:supplier_id>/delete",
methods=["POST"])
@login_required
def delete_supplier(supplier_id):
    conn = get_db_connection()
    try:
        conn.execute("DELETE FROM suppliers WHERE supplier_id =
?", (supplier_id,))
        conn.commit()
        flash("Supplier deleted successfully.", "success")
    except sqlite3.IntegrityError:
        flash("This supplier is used by products. Delete related products
first.",
"danger")
    finally:
        conn.close()
    return redirect(url_for("suppliers"))
```

Security Engineering Note: Session handling is managed via unique cryptographic tokens signed by the server's private key. This ensures warehouse actions cannot be modified or forged by outside clients.

5. CONCLUSION

The Textile Warehouse Stock Management System effectively eliminates the operational risks and data silos common in legacy inventory setups. By combining a 3NF relational database structure with a clean web interface, the system ensures data consistency while offering a direct, human-readable view of operations. Built-in features like atomic stock level enforcement and automatic transaction logging prevent invalid stock states and protect data integrity. This framework provides an ideal foundation for future expansions, such as automated inventory forecasting, barcode integration, and enterprise resource planning (ERP) sync tools.

